

Design Engineering

1

1

What is software design

- Design is a meaningful engineering representation of something that is to be built. It can be traced to a customer's requirements and at the same time assessed for quality against a set of predefined criteria for "good" design.
- In the software engineering context, design focuses on four major areas of concern: data, architecture, interfaces, and components.

2

2

Focused area of software design

- The *data design* transforms the information domain model created during analysis into the data structures that will be required to implement the software. The data objects and relationships defined in the entity relationship diagram and the detailed data content depicted in the data dictionary provide the basis for the data design activity.

3

3

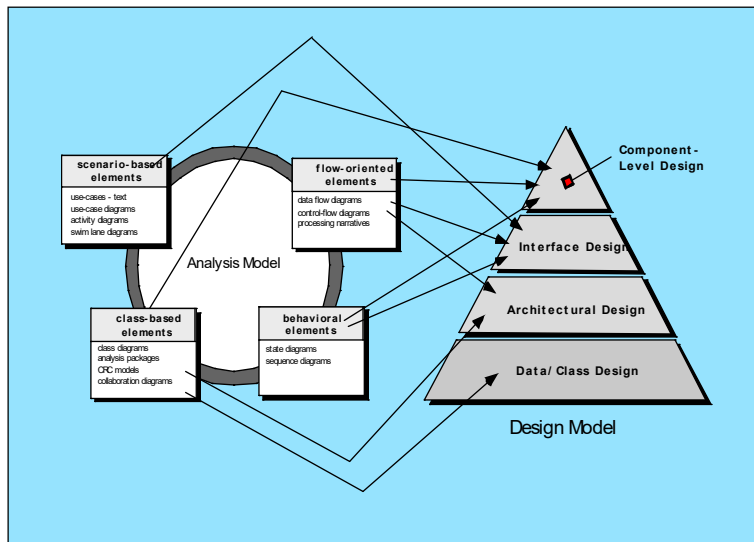
Focused area of software design

- The *architectural design* defines the relationship between major structural elements of the software.
- The *interface design* describes how the software communicates within itself, with systems that interoperate with it, and with humans who use it.
- The *component-level design* transforms structural elements of the software architecture into a procedural description of software components.
 - A component is a fairly independent piece of program that has a name, some instructions and some data of its own. (method, class, package)

4

4

Analysis Model -> Design Model



5

5

Design and Quality

- the design must implement all of the explicit requirements contained in the analysis model, and it must accommodate all of the implicit requirements desired by the customer.
- the design must be a readable, understandable guide for those who generate code and for those who test and subsequently support the software.
- the design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective.

6

6

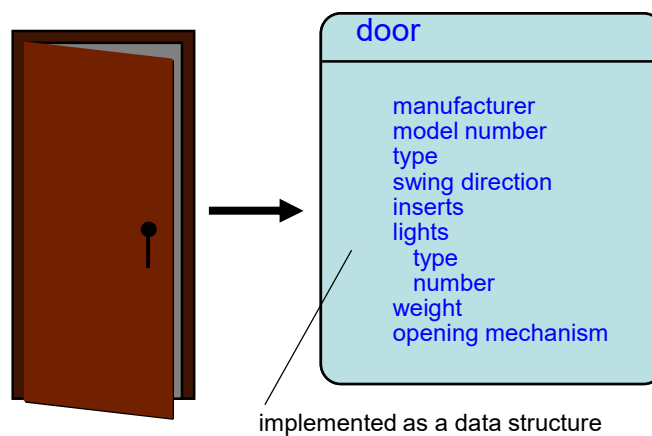
Fundamental Concepts

- **abstraction**—data, procedure, control
- **architecture**—the overall structure of the software
- **patterns**—"conveys the essence" of a proven design solution
- **modularity**—compartmentalization of data and function, easier to build, easier to change, easier to fix
- **hiding**—controlled interfaces
- **Functional independence**—single-minded function and low coupling
- **refinement**—elaboration of detail for all abstractions
- **Refactoring**—a reorganization technique that simplifies the design

7

7

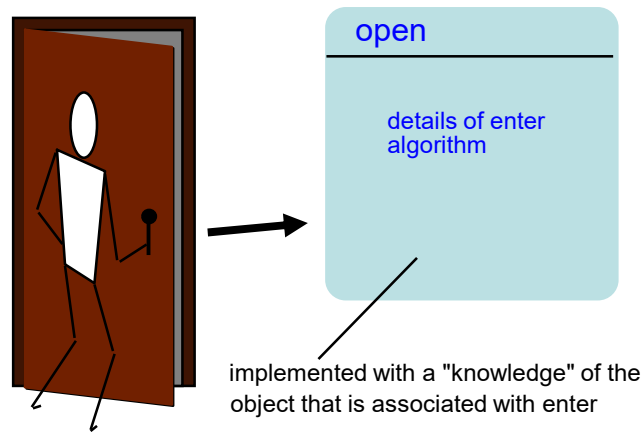
Data Abstraction



8

8

Procedural Abstraction



9

9

Architecture

"The overall structure of the software and the ways in which that structure provides conceptual integrity for a system." [SHA95a]

- **Structural properties.** This aspect of the architectural design representation defines the components of a system (e.g., modules, objects, filters) and the manner in which those components are packaged and interact with one another. For example, objects are packaged to encapsulate both data and the processing that manipulates the data and interact via the invocation of methods
- **Extra-functional properties.** The architectural design description should address how the design architecture achieves requirements for performance, capacity, reliability, security, adaptability, and other system characteristics.
- **Families of related systems.** The architectural design should draw upon repeatable patterns that are commonly encountered in the design of families of similar systems. In essence, the design should have the ability to reuse architectural building blocks.

10

10

Functional independence

- Functional independence
 - Independent modules are easier to maintain (and test) because secondary effects caused by design or code modification are limited, error propagation is reduced, and reusable modules are possible.
- Cohesion
 - A cohesive module performs a single task within a software procedure, requiring little interaction with procedures being performed in other parts of a program or the module should do just one thing

11

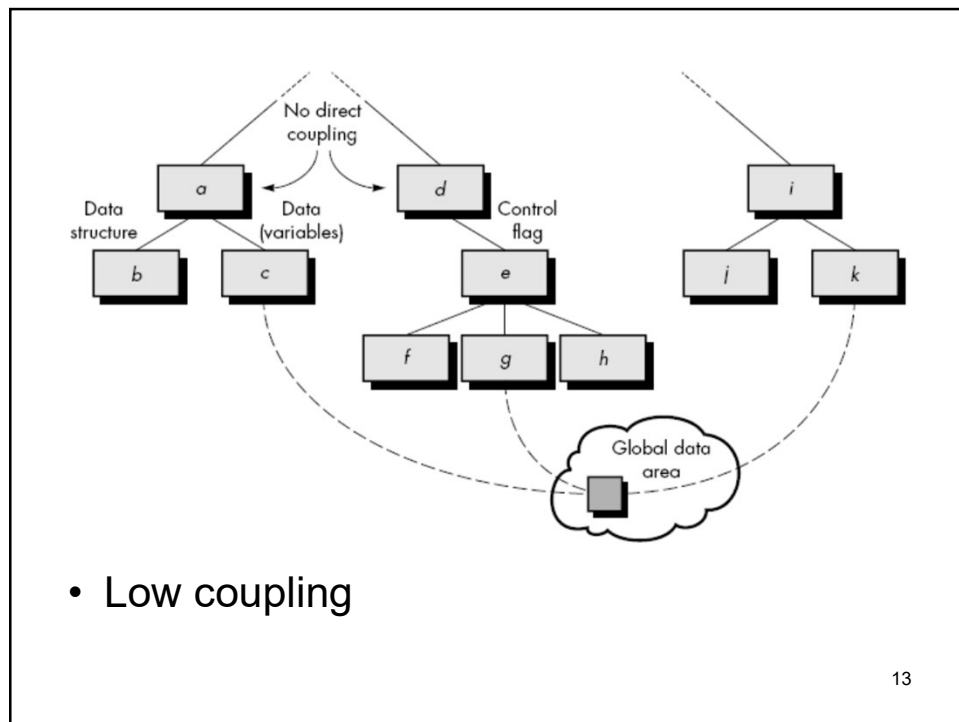
11

Functional independence

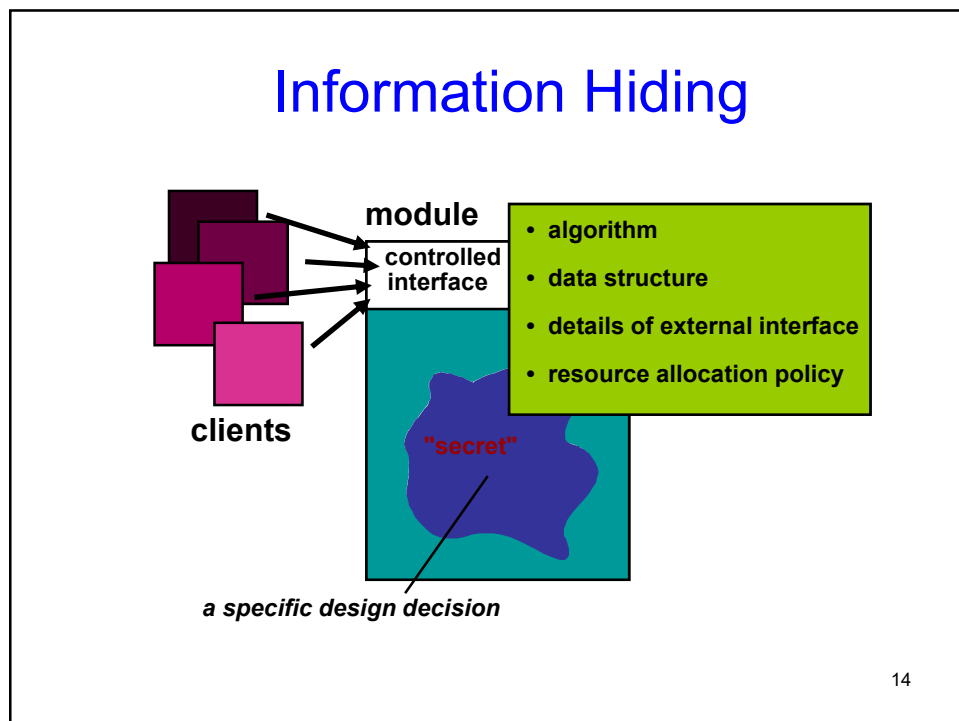
- High cohesion
- Coupling
 - Coupling is a measure of interconnection among modules in a software structure. Coupling depends on the interface complexity between modules, the point at which entry or reference is made to a module, and what data pass across the interface.
 - Data coupling, Stamp coupling, Control coupling, External coupling, Common coupling, i.e.

12

12



13



14

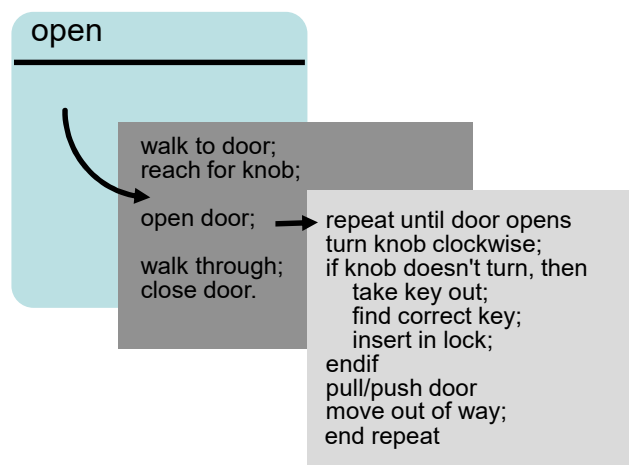
Why Information Hiding?

- reduces the likelihood of “side effects”
- limits the global impact of local design decisions
- emphasizes communication through controlled interfaces
- discourages the use of global data
- leads to encapsulation—an attribute of high quality design
- results in higher quality software

15

15

Stepwise Refinement



16

16

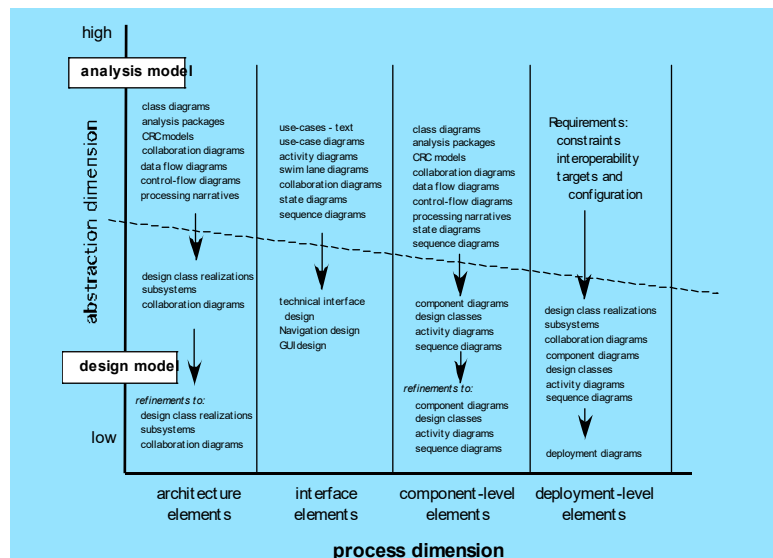
Refactoring

- Fowler [FOW99] defines refactoring in the following manner:
 - "Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code [design] yet improves its internal structure."
- When software is refactored, the existing design is examined for
 - redundancy
 - unused design elements
 - inefficient or unnecessary algorithms
 - poorly constructed or inappropriate data structures
 - or any other design failure that can be corrected to yield a better design.

17

17

The Design Model



18

18

Design Model Elements

- **Data elements**
 - Data model --> data structures
 - Data model --> database architecture
- **Architectural elements**
 - Application domain
 - Analysis classes, their relationships, collaborations and behaviors are transformed into design realizations
 - Patterns and “styles”
- **Interface elements**
 - the user interface (UI)
 - external interfaces to other systems, devices, networks or other producers or consumers of information
 - internal interfaces between various design components.

19

19

Design Model Elements

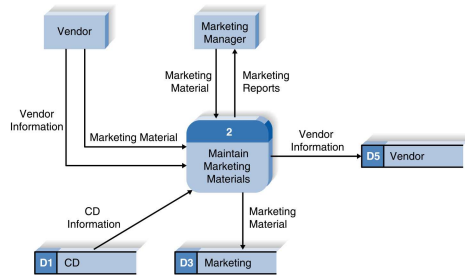
- **Component elements**
 - Data structures for all local data objects and algorithmic design for all processing that occurs within a component and an interface that allows access to all component operations (behaviors)
- **Deployment elements**
 - How software functionality and subsystems will be allocated with in the physical computing environment that will support the software.

20

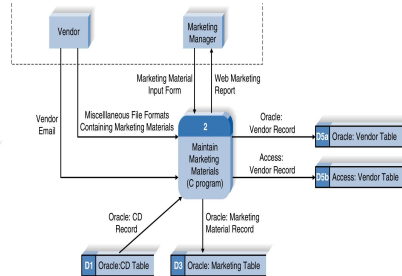
20

Contrasting Logical and Physical DFDs

A Logical DFD



A Physical DFD



21

21

Q&A

22

22